

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

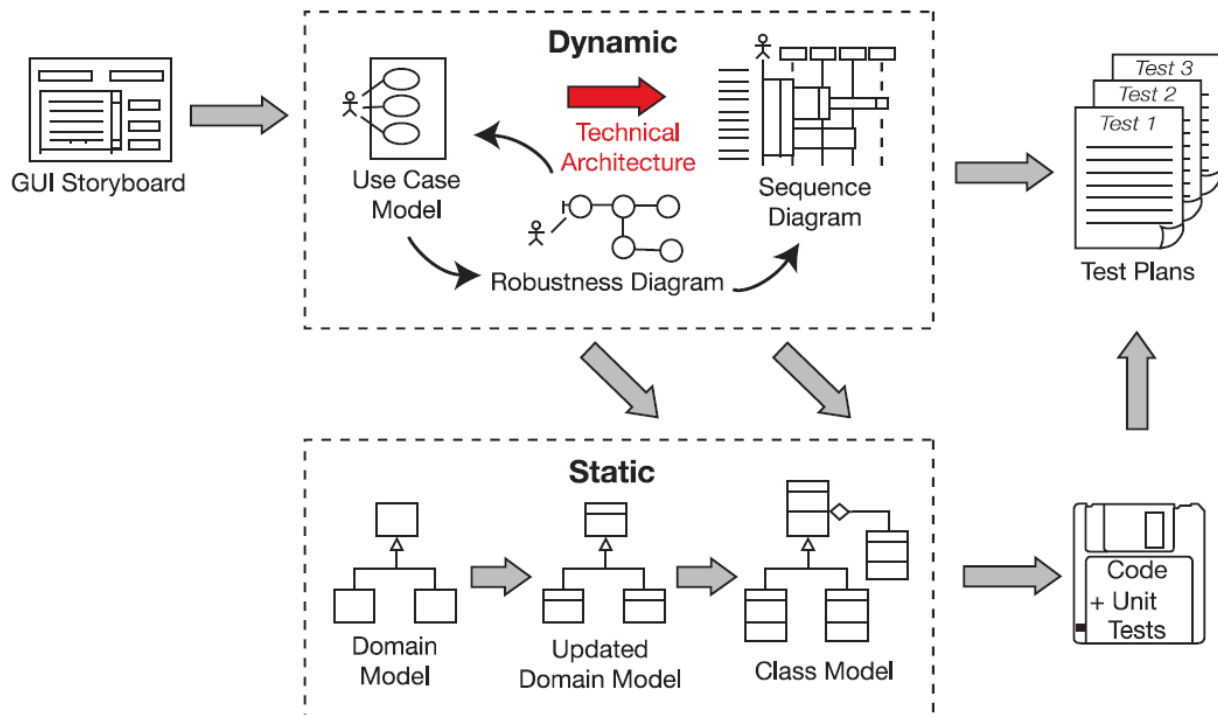
Nội dung chính

Bài này giới thiệu vai trò của Technical Architecture (TA) trong quy trình phát triển phần mềm theo ICONIX, như một bước chuyển tiếp quan trọng từ phân tích sang thiết kế chi tiết. Nội dung tập trung làm rõ lý do tại sao dự án cần có một kiến trúc kỹ thuật rõ ràng, thống nhất để tránh tình trạng thiết kế rời rạc, khó bảo trì và kiểm thử. Bài học cũng trình bày các nguyên tắc cơ bản khi xây dựng TA, mô hình phân tầng kiến trúc, cũng như các lỗi kiến trúc thường gặp. Qua ví dụ Internet Bookstore, sinh viên sẽ hiểu cách TA hỗ trợ việc hiện thực hóa các mô hình phân tích thành một hệ thống kỹ thuật cụ thể. Đây là bước nền tảng để chuẩn bị cho giai đoạn sequence diagram và coding sau này.

Mục tiêu cần đạt

Sau khi học xong bài này, sinh viên có thể:

- Hiểu được vai trò và vị trí của Technical Architecture trong quy trình ICONIX.
- Trình bày được các nguyên tắc và mô hình cơ bản khi thiết kế kiến trúc kỹ thuật.
- Phân tích và áp dụng kiến trúc kỹ thuật vào một hệ thống mẫu (Internet Bookstore).
- Nhận diện và tránh các lỗi kiến trúc phổ biến, chuẩn bị nền tảng cho thiết kế chi tiết.



7.1 Định nghĩa và vai trò của Technical Architecture trong ICONIX

Trong quy trình ICONIX, Technical Architecture (TA) được xem như một bản thiết kế kỹ thuật tổng thể, xác định cấu trúc tổ chức của hệ thống phần mềm trước khi bước sang giai đoạn thiết kế chi tiết và lập trình. Nếu như Domain Model tập trung vào các khái niệm nghiệp vụ, Use Case mô tả hành vi người dùng, và Robustness Diagram bắc cầu giữa hành vi – đối tượng, thì Technical Architecture chính là nơi những yếu tố này được “đóng khung” trong một kiến trúc thực thi cụ thể, định hình cách mà hệ thống sẽ được xây dựng trên thực tế. Đây chính là giai đoạn chuyển từ “what” sang “how” theo nghĩa kỹ thuật.

Điểm đặc biệt là ICONIX không xem TA như một bước thiết kế tự do, mà là một phần liên mạch trong tiến trình phát triển. Sau khi nhóm hoàn thành Preliminary Design Review (PDR), toàn bộ Use Case và mô hình đã được kiểm duyệt logic và thống nhất. Tuy nhiên, các mô hình này vẫn còn ở mức trừu tượng — chúng nói “hệ thống cần làm gì”, nhưng chưa nói rõ “các thành phần kỹ thuật sẽ hợp tác với nhau như thế nào”. TA xuất hiện đúng lúc để trả lời câu hỏi đó: nó quy định cấu trúc tầng, xác định những component chính (UI, controller, business logic, persistence), cách chúng tương tác, cũng như những ràng buộc kỹ thuật sẽ được áp dụng xuyên suốt dự án.

Ở cấp độ tổ chức dự án, TA đóng vai trò như một hợp đồng kiến trúc giữa các thành viên. Khi nhiều lập trình viên cùng phát triển, việc có một kiến trúc thống nhất sẽ giúp họ tuân theo cùng một “bản đồ” thay vì mỗi người thiết kế theo cảm tính riêng. Điều này đặc biệt quan trọng trong các dự án lớn hoặc khi sử dụng framework phức tạp như Spring, .NET hay microservices. Một Technical Architecture được thiết kế tốt giúp giảm xung đột trong code, tránh chồng chéo trách nhiệm giữa các lớp, và tạo điều kiện cho kiểm thử tự động cũng như mở rộng về sau.

Một vai trò quan trọng khác của TA là giảm thiểu rủi ro kỹ thuật sớm. Ở giai đoạn này, nhóm có thể xác định những điểm có nguy cơ gây tắc nghẽn (ví dụ: giao tiếp giữa client và server, cơ chế bảo mật, khả năng mở rộng cơ sở dữ liệu), từ đó đưa ra quyết định kiến trúc trước khi hàng nghìn dòng code được viết ra. Đây là cách tiếp cận “phòng ngừa chủ động” thay vì “chữa cháy” khi lỗi đã xảy ra ở giai đoạn triển khai. ICONIX nhấn mạnh rằng TA không cần phải chi tiết đến mức class diagram, nhưng phải đủ rõ để định hướng toàn bộ thiết kế.

7.2 Top 10 Guidelines về Technical Architecture

Khi bước vào thiết kế kiến trúc kỹ thuật, các nhóm phát triển thường rơi vào hai thái cực: hoặc quá hời hợt, chỉ vẽ vài sơ đồ tầng sơ sài rồi lao vào lập trình; hoặc quá cầu kỳ, tạo ra những bản kiến trúc phức tạp, rườm rà mà ít giá trị thực tiễn. ICONIX chủ trương một cách tiếp cận trung dung: kiến trúc phải đủ đơn giản để dễ hiểu, nhưng đủ chặt chẽ để dẫn dắt toàn bộ giai đoạn thiết kế và triển khai. Để đạt được điều này, họ đề xuất mười nguyên tắc thực hành (Top 10 Guidelines) đóng vai trò như một “khung tư duy” khi xây dựng Technical Architecture.

Nguyên tắc đầu tiên là giữ kiến trúc càng đơn giản càng tốt nhưng không đơn giản hóa quá mức. Kiến trúc kỹ thuật không cần cầu kỳ để tỏ ra “cao cấp”; quan trọng là tính rõ ràng, dễ hiểu và phù hợp với yêu cầu thực tế. Một kiến trúc quá phức tạp sẽ tạo gánh nặng bảo trì, trong khi một kiến trúc quá sơ sài sẽ thiếu định hướng cho nhóm phát triển. Ví dụ, Internet Bookstore có thể vận hành hiệu quả chỉ với kiến trúc ba tầng (presentation, business, persistence) mà không cần microservices. Tuy nhiên, nếu gộp mọi logic vào JSP, hệ thống sẽ nhanh chóng trở nên rối rắm, khó kiểm thử và mở rộng.

Thứ hai, cần phân tách rõ ràng các trách nhiệm giữa các tầng kiến trúc. Presentation chỉ nên tập trung vào giao tiếp với người dùng, business xử lý logic nghiệp vụ, còn persistence đảm nhiệm truy xuất dữ liệu. Việc chồng chéo vai trò giữa các tầng — như nhúng SQL vào giao diện hay đưa logic nghiệp vụ xuống DAO — là lỗi phổ biến gây ra sự lẫn lộn trách nhiệm. Trong Internet Bookstore, nếu BookDetailsPage.jsp trực tiếp truy vấn database mà không thông qua controller và DAO, điều này đã vi phạm kiến trúc tầng và sẽ gây khó khăn khi kiểm thử hoặc bảo trì.

Thứ ba, sử dụng sơ đồ thành phần (component diagram) và sơ đồ triển khai (deployment diagram) như công cụ giao tiếp kiến trúc. Các sơ đồ này cung cấp cái nhìn trực quan giúp nhóm phát triển và các bên liên quan hiểu được cách hệ thống được tổ chức mà không cần đi sâu vào chi tiết code. Component diagram chỉ ra các khối chính (UI, controller, service, DAO, external API...), còn deployment diagram thể hiện cách các thành phần này được bố trí trên hạ tầng vật lý. Trong Internet Bookstore, hai loại sơ đồ này có thể minh họa rõ ràng luồng giao tiếp từ JSP → Spring Controller → Service → DAO → Database.

Thứ tư, liên kết kiến trúc với các quyết định kỹ thuật thực tế. Kiến trúc không tồn tại trong chân không mà phải phản ánh các lựa chọn framework, công nghệ và ràng buộc hạ tầng. Nếu nhóm dùng Spring MVC, kiến trúc phải thể hiện rõ vai trò của

DispatcherServlet, controller và service. Nếu hệ thống xây dựng theo REST, cần mô tả các endpoint chính, session handling và cơ chế bảo mật. Internet Bookstore có thể chọn Spring MVC + Hibernate làm nền tảng, và các quyết định này phải được phản ánh rõ trong kiến trúc để tránh tình trạng “vẽ một đằng, code một nẻo”.

Thứ năm, kiến trúc kỹ thuật cần được chia sẻ và kiểm duyệt sớm trong nhóm. Một bản kiến trúc tốt không có giá trị nếu chỉ nằm trên giấy mà không được mọi thành viên hiểu và tuân thủ. ICONIX khuyến nghị tổ chức các buổi “architecture walkthrough” tương tự như PDR, nơi nhóm cùng rà soát sơ đồ thành phần, luồng dữ liệu và các quyết định chính. Đây là cơ hội phát hiện mâu thuẫn, thiếu sót và đảm bảo toàn đội cùng đi theo một định hướng kỹ thuật thống nhất.

Thứ sáu, tính kiểm thử (testability) phải được quan tâm ngay từ kiến trúc. Một kiến trúc tốt không chỉ giúp hệ thống chạy được mà còn phải tạo điều kiện cho việc kiểm thử hiệu quả. Việc tách biệt rõ ràng giữa controller, service và DAO cho phép viết unit test dễ dàng bằng cách mock các thành phần phụ thuộc. Nếu kiến trúc gắn chặt tất cả vào một khối không tách rời, việc kiểm thử sẽ trở nên phức tạp hoặc không khả thi. Internet Bookstore khi tách đúng các tầng có thể dễ dàng kiểm thử logic xác thực, kiểm duyệt review hay xử lý giỏ hàng một cách độc lập.

Thứ bảy, các khía cạnh phi chức năng như bảo mật (security) và khả năng mở rộng (scalability) cần được xem xét từ đầu. Đây là điểm mà nhiều nhóm sinh viên thường bỏ qua. Nếu hệ thống có cơ chế xác thực phức tạp, kiến trúc phải xác định rõ nơi xử lý, cách bảo vệ dữ liệu, và kiểm soát truy cập ở tầng nào. Tương tự, nếu kỳ vọng số lượng người dùng lớn, kiến trúc cần tính đến caching, load balancing hoặc khả năng nhân bản tầng web. Việc “chữa cháy” những vấn đề này ở cuối dự án thường rất tốn kém.

Thứ tám, đảm bảo mỗi use case có một “đường đi rõ ràng” xuyên qua kiến trúc. Từ boundary (UI) → controller → business logic → persistence, luồng xử lý của từng use case phải được phản ánh nhất quán trong kiến trúc. Điều này giúp việc thiết kế sequence diagram trở nên dễ dàng và tránh trường hợp kiến trúc “đẹp trên giấy” nhưng không mapping được vào luồng thực tế. Khi kiểm tra use case “Write Customer Review”, ta có thể lần theo đường đi rõ ràng qua các tầng: trang nhập review → controller xử lý → service validate và queue → DAO ghi vào database.

Thứ chín, tránh trộn lẫn quá nhiều công nghệ hoặc pattern cùng lúc. Một sai lầm phổ biến là cố gắng kết hợp nhiều framework, pattern, hoặc kiến trúc (ví dụ vừa dùng MVC, vừa microservices, vừa event-driven) mà không có lý do rõ ràng. Kết quả là hệ thống trở nên rối rắm, khó bảo trì và khó onboard thành viên mới. ICONIX khuyên chọn một kiến trúc cốt lõi phù hợp với dự án, rồi triển khai nhất quán xuyên suốt, thay vì “tham” nhiều công nghệ cho có vẻ hiện đại.

Cuối cùng, Technical Architecture cần có tính “sống” — được cập nhật khi yêu cầu hoặc mô hình thay đổi. Kiến trúc không phải là một bản thiết kế chết nằm yên từ đầu đến cuối dự án. Khi use case hoặc domain model thay đổi đáng kể, kiến trúc cũng

phải phản ánh lại để đảm bảo tính nhất quán. Việc duy trì kiến trúc như một artefact sống giúp nhóm tránh rơi vào tình trạng “documentation drift” — khi tài liệu kiến trúc không còn phản ánh thực tế code.

7.3 Architectural Layering — Mô hình phân tầng kiến trúc

Một trong những quyết định kỹ thuật quan trọng nhất trong bất kỳ hệ thống phần mềm nào là xác định mô hình phân tầng (layering). Phân tầng kiến trúc giúp tách biệt rõ ràng các trách nhiệm, kiểm soát luồng dữ liệu, và tạo ra những ranh giới tự nhiên để hỗ trợ kiểm thử, bảo trì, cũng như phát triển song song. Nếu không có kiến trúc phân tầng rõ ràng, hệ thống rất dễ rơi vào tình trạng “spaghetti code” — nơi giao diện, logic nghiệp vụ và truy xuất dữ liệu lẫn lộn, khiến mỗi thay đổi nhỏ cũng có thể gây hiệu ứng dây chuyền khó kiểm soát.

Trong quy trình ICONIX, phân tầng kiến trúc đóng vai trò bắc cầu giữa Robustness Diagram và Sequence Diagram. Ở bước Robustness, chúng ta đã xác định boundary, controller và entity — đây chính là “gợi ý tự nhiên” cho việc thiết kế các tầng kiến trúc tương ứng: boundary thường ánh xạ vào tầng presentation, controller thành tầng logic/điều phối, còn entity tương ứng với tầng persistence. Khi bắt đầu sequence diagram, việc đã có một kiến trúc phân tầng rõ ràng sẽ giúp định hình ngay đường đi của thông điệp giữa các đối tượng, tránh thiết kế tùy tiện.

Mô hình phổ biến nhất và cũng là ví dụ chuẩn trong ICONIX là kiến trúc ba tầng (three-tier architecture) gồm:

- Presentation Layer: xử lý giao tiếp với người dùng hoặc hệ thống bên ngoài, thường bao gồm UI, web pages, form nhập liệu, API endpoints.
- Business Layer: xử lý logic nghiệp vụ, điều phối luồng xử lý giữa UI và dữ liệu, thường bao gồm controller và service classes.
- Persistence Layer: truy xuất và lưu trữ dữ liệu, thường gồm DAO (Data Access Object), repository hoặc ORM mapping.

Internet Bookstore là ví dụ điển hình cho mô hình này. Khi người dùng truy cập trang BookDetailsPage, tầng presentation (JSP) sẽ nhận yêu cầu, chuyển đến controller ở tầng business (DisplayBookDetailsController), controller gọi BookService để xử lý logic, sau đó service tương tác với BookDAO ở tầng persistence để lấy dữ liệu từ database. Kết quả được trả ngược lại qua các tầng để hiển thị cho người dùng. Chuỗi tương tác này thể hiện rõ đường đi của một use case xuyên qua các tầng kiến trúc, phản ánh đúng mô hình MVC và giúp quá trình thiết kế sequence diagram trở nên gần như “cơ học”.

Một lợi ích lớn của kiến trúc phân tầng là tăng tính mô-đun và khả năng kiểm thử độc lập. Khi các tầng được tách biệt rõ, ta có thể dễ dàng kiểm thử business layer bằng cách mock tầng persistence, hoặc kiểm thử UI mà không phụ thuộc vào logic bên dưới. Điều này đặc biệt quan trọng trong các dự án lớn hoặc khi áp dụng CI/CD, nơi việc tự động hóa kiểm thử là yếu tố then chốt. Ngoài ra, phân tầng còn giúp nhóm có

thể làm việc song song: một nhóm phát triển UI, nhóm khác xây service, nhóm khác thiết kế cơ sở dữ liệu, miễn là giao diện giữa các tầng được định nghĩa rõ ràng.

Tuy nhiên, ICONIX cũng cảnh báo rằng phân tầng không nên trở thành mục tiêu hình thức. Một số nhóm sinh viên thường mắc lỗi “chia tầng cho có”, nhưng bên trong lại để logic rò rỉ qua các tầng, hoặc tạo ra quá nhiều tầng nhỏ vụn khiến hệ thống trở nên phức tạp không cần thiết. Việc thêm một tầng chỉ có ý nghĩa nếu nó thực sự mang lại lợi ích về kiểm thử, tái sử dụng hoặc bảo trì. Một kiến trúc có 5–6 tầng nhưng các tầng mờ nhạt, không có ranh giới trách nhiệm rõ ràng thì không bằng một kiến trúc ba tầng gọn gàng mà nhất quán.

Để mô hình hóa kiến trúc phân tầng, ICONIX khuyến nghị sử dụng UML Component Diagram. Mỗi tầng được thể hiện như một nhóm thành phần (component) có ràng buộc phụ thuộc rõ ràng. Presentation phụ thuộc vào Business, Business phụ thuộc vào Persistence, và không có quan hệ ngược chiều trực tiếp. UML cho phép dùng các cổng (ports) và giao diện (interfaces) để thể hiện cách các tầng giao tiếp với nhau. Trong Internet Bookstore, ta có thể mô hình hóa: JSP → Spring Controller → Service → DAO → Database, với các dependency rõ ràng theo chiều từ trên xuống dưới. Đây là mô hình cơ bản nhưng cực kỳ hiệu quả, đặc biệt khi chuyển sang thiết kế sequence diagram cho từng use case.

Một điểm quan trọng nữa là tuân thủ nguyên tắc phụ thuộc một chiều giữa các tầng. Presentation có thể gọi xuống Business, Business gọi xuống Persistence, nhưng tầng dưới không được “ngược dòng” gọi lên tầng trên. Vi phạm nguyên tắc này là một trong những nguồn gốc chính của coupling rối rắm và khó kiểm thử. ICONIX nhấn mạnh rằng mỗi tầng cần có giao diện rõ ràng, che giấu chi tiết triển khai bên trong, để các tầng khác chỉ cần biết “nó làm gì” chứ không cần biết “nó làm như thế nào”.

7.4 Ví dụ Technical Architecture: Internet Bookstore

Để minh họa cho các nguyên tắc kiến trúc kỹ thuật, ICONIX sử dụng hệ thống Internet Bookstore như một ví dụ xuyên suốt. Đây là một hệ thống đủ quen thuộc, có nhiều use case điển hình (đăng nhập, xem thông tin sách, viết đánh giá, xử lý giỏ hàng...), và có độ phức tạp vừa phải để thể hiện kiến trúc tầng rõ ràng mà không bị sa vào chi tiết kỹ thuật quá mức. Giả sử nhóm phát triển quyết định sử dụng Spring MVC làm framework phía server và JDBC/Hibernate cho tầng truy xuất dữ liệu. Dưới đây là cách kiến trúc được tổ chức.

Ở tầng Presentation, hệ thống sử dụng các trang JSP và Spring MVC Controller để xử lý yêu cầu từ phía người dùng. Mỗi use case tương ứng với một màn hình và một controller riêng. Ví dụ, use case “Show Book Details” có BookDetailsPage.jsp làm UI, và BookDetailsController nhận các request GET/POST, gọi xuống tầng dưới để lấy dữ liệu và chuẩn bị model cho JSP render. Các boundary class từ sơ đồ robustness (như LoginPage, BookDetailsPage, WriteReviewPage) ánh xạ gần như trực tiếp thành các trang JSP hoặc endpoint giao tiếp REST.

Tầng Business là nơi xử lý logic nghiệp vụ. Mỗi use case quan trọng có thể có một hoặc nhiều service class tương ứng. Ví dụ, khi người dùng yêu cầu xem chi tiết sách, BookDetailsController sẽ gọi BookService, lớp này chịu trách nhiệm kiểm tra trạng thái sách, tổng hợp đánh giá trung bình, lấy thông tin mô tả và xử lý các tình huống đặc biệt (như sách bị ẩn hoặc không tồn tại). Trong use case “Write Customer Review”, ReviewService sẽ thực hiện validate độ dài review, kiểm tra rating, gửi review vào hàng đợi kiểm duyệt, và thông báo lại cho người dùng. Đây chính là nơi các controller từ sơ đồ robustness được ánh xạ thành các service thực thi logic thực sự.

Tầng Persistence bao gồm các lớp DAO (Data Access Object) chịu trách nhiệm tương tác với cơ sở dữ liệu. BookDAO cung cấp các phương thức như findBookByISBN(), getReviewsForBook(), trong khi ReviewDAO đảm nhiệm việc lưu và truy xuất review. DAO đóng vai trò là lớp trung gian, tách biệt hoàn toàn business logic khỏi chi tiết truy xuất dữ liệu. Điều này cho phép thay đổi công nghệ lưu trữ (ví dụ từ JDBC sang Hibernate, hoặc từ SQL sang NoSQL) mà không ảnh hưởng đến tầng business. Nó cũng hỗ trợ unit testing dễ dàng, vì có thể mock DAO khi kiểm thử service.

Để mô hình hóa kiến trúc này, ICONIX sử dụng UML Component Diagram. Trong sơ đồ, Presentation Layer bao gồm các JSP và Controller; Business Layer gồm các Service class; Persistence Layer gồm các DAO và Database. Các mũi tên phụ thuộc thể hiện luồng giao tiếp: Presentation gọi Business, Business gọi Persistence. Không có quan hệ ngược chiều, đảm bảo tính tách biệt và giảm coupling. Deployment Diagram thì thể hiện các component này chạy trên những node vật lý nào: Web Server cho JSP/Controller, Application Server cho Business Logic, và Database Server cho Persistence. Mô hình này đơn giản nhưng rõ ràng, dễ mở rộng và kiểm thử.

Luồng xử lý một request tiêu biểu minh họa kiến trúc này có thể thấy rõ qua use case “Show Book Details”. Người dùng nhấn vào một đường link trên Home Page → request được gửi tới BookDetailsController → controller gọi BookService để lấy dữ liệu → service gọi BookDAO để truy vấn database → kết quả trả ngược lên controller → controller đẩy dữ liệu vào model và forward đến BookDetailsPage.jsp để hiển thị. Toàn bộ quá trình này đi đúng một đường tuyến tính xuyên qua các tầng, thể hiện rõ nguyên tắc architectural layering. Đây chính là “đường đi” của use case mà ICONIX luôn muốn đảm bảo khi thiết kế kiến trúc kỹ thuật.

Cuối cùng, kiến trúc này còn thể hiện rõ các điểm mở rộng (extension points) cho tương lai. Nếu muốn bổ sung API REST cho mobile app, có thể thêm các REST controller song song với JSP mà không ảnh hưởng đến tầng business và persistence. Nếu cần thay đổi hệ quản trị cơ sở dữ liệu, chỉ cần sửa các lớp DAO. Nếu cần caching để tăng hiệu năng, có thể thêm một caching layer giữa business và persistence mà không phá vỡ cấu trúc hiện có. Đây là minh chứng cụ thể cho tính linh hoạt mà một kiến trúc kỹ thuật tốt mang lại.

Một điểm quan trọng giúp kiến trúc ba tầng của Internet Bookstore vận hành trơn tru là việc áp dụng các nguyên tắc Inversion of Control (IoC) và Dependency Injection (DI). Đây là hai cơ chế cốt lõi trong các framework hiện đại như Spring, đóng vai trò

giảm sự phụ thuộc cứng (tight coupling) giữa các lớp, từ đó làm cho kiến trúc trở nên linh hoạt, kiểm thử được và dễ mở rộng.

Trong cách thiết kế truyền thống, các lớp thường tự khởi tạo hoặc tìm kiếm các phụ thuộc mà chúng cần sử dụng. Ví dụ, một `BookService` có thể trực tiếp tạo ra một `new BookDAO()` để truy xuất dữ liệu. Điều này tạo ra sự gắn chặt giữa hai lớp: nếu ta muốn thay thế DAO khác hoặc viết unit test cho `BookService`, ta buộc phải sửa code bên trong lớp. IoC đảo ngược mối quan hệ này: thay vì để các lớp tự quản lý phụ thuộc, một container trung tâm (như Spring IoC Container) sẽ chịu trách nhiệm tạo, quản lý và “bơm” (inject) các phụ thuộc vào đúng nơi cần. Nhờ đó, các lớp chỉ cần khai báo “tôi cần cái gì”, còn việc cung cấp sẽ do kiến trúc đảm nhiệm.

Dependency Injection là kỹ thuật cụ thể để hiện thực hóa IoC. Thay vì `BookService` tự khởi tạo `BookDAO`, ta định nghĩa `BookDAO` như một dependency và inject nó vào `BookService` thông qua constructor hoặc setter. Spring sẽ đọc cấu hình (XML, annotation, Java config) để biết cách khởi tạo các bean và “nối dây” chúng lại với nhau. Khi `BookService` cần dùng DAO, nó chỉ việc gọi phương thức đã được inject sẵn. Điều này giúp lớp `BookService` trở nên độc lập với việc tạo DAO, có thể dễ dàng thay thế bằng một mock DAO trong unit test, hoặc đổi sang một implementation khác mà không cần sửa code.

Trong kiến trúc Internet Bookstore, IoC/DI đóng vai trò kết nối tự động giữa các tầng:

- Các Controller được Spring khởi tạo và inject Service tương ứng.
- Các Service nhận DAO qua injection, không khởi tạo trực tiếp.
- DAO được Spring quản lý như các bean, có thể thay đổi hoặc mock tùy vào môi trường (dev/test/prod).

Nhờ IoC/DI, kiến trúc layering không chỉ nằm trên lý thuyết mà được hiện thực hóa đúng theo nguyên tắc phụ thuộc một chiều: `Presentation` → `Business` → `Persistence`. Không có tầng nào phải “tự tìm” tầng khác, cũng không có các `new()` ngổn ngang rải rác trong code. Hơn nữa, việc kiểm thử unit trở nên cực kỳ dễ dàng, vì ta có thể inject các mock object vào service/controller mà không cần đụng vào cấu hình hệ thống.

7.5 Testability & Security Considerations

Một kiến trúc kỹ thuật vững chắc không chỉ nhằm tổ chức code cho gọn gàng mà còn đóng vai trò tạo điều kiện cho kiểm thử và bảo mật trở nên hiệu quả, có hệ thống. ICONIX xem đây là hai khía cạnh bắt buộc phải được suy nghĩ từ giai đoạn Technical Architecture, vì nếu bỏ qua ở thời điểm này, hệ thống sẽ đối mặt với những ràng buộc cứng nhắc và lỗi hổng khó khắc phục khi đã đi vào thiết kế chi tiết hoặc lập trình.

1. Testability — khả năng kiểm thử bắt nguồn từ kiến trúc

Testability không thể “vá” sau khi kiến trúc đã hình thành. Nó là kết quả trực tiếp của những quyết định về phân tầng, về cách các thành phần tương tác và về mức độ coupling giữa chúng. Một kiến trúc có tầng rõ ràng, có giao diện tách biệt và tuân thủ

nguyên tắc dependency injection sẽ cho phép kiểm thử unit dễ dàng, viết integration test linh hoạt và giảm phụ thuộc vào môi trường thực.

Ví dụ, trong Internet Bookstore, việc tách rõ Presentation, Business và Persistence Layer cùng với cơ chế IoC/DI giúp các lớp business có thể được kiểm thử độc lập. BookService có thể được unit test bằng cách inject một mock BookDAO, kiểm tra logic xử lý mà không cần kết nối thật tới database. Tương tự, controller có thể được test bằng cách mock service bên dưới, hoặc kiểm thử integration bằng Spring Test mà không cần khởi động toàn bộ server. Nếu các tầng bị trộn lẫn, ví dụ controller tự khởi tạo DAO, thì việc viết unit test gần như không thể, vì mỗi test sẽ phụ thuộc vào cả hệ thống bên dưới.

ICONIX khuyến nghị mỗi use case quan trọng nên được thiết kế sao cho đường đi của thông điệp xuyên qua các tầng có thể được kiểm thử một cách độc lập theo từng đoạn: test logic nghiệp vụ mà không cần UI, test truy xuất dữ liệu mà không cần controller, test UI mà không cần backend thật. Đây là cách tiếp cận “thiết kế để kiểm thử” (design for testability), và kiến trúc là nơi đặt nền móng cho khả năng đó. IoC/DI, interface rõ ràng, và dependency một chiều là ba yếu tố kỹ thuật then chốt đảm bảo testability tốt.

Một lợi ích khác của kiến trúc kiểm thử tốt là nó giúp giảm rủi ro kỹ thuật sớm. Thay vì chờ đến cuối chu trình phát triển mới phát hiện lỗi nghiêm trọng, nhóm có thể viết test từ rất sớm, chạy tự động trong CI/CD pipeline và phát hiện vấn đề ngay khi một tầng bị thay đổi bất ngờ. Đây chính là “lá chắn đầu tiên” giúp hệ thống ổn định và dễ bảo trì.

2. Security — bảo mật bắt đầu từ kiến trúc

Cũng như testability, bảo mật không thể “vá” ở cuối dự án. Một kiến trúc kỹ thuật lỏng lẻo, không có ranh giới rõ ràng giữa các tầng sẽ khiến việc kiểm soát truy cập, kiểm tra dữ liệu đầu vào, và bảo vệ tài nguyên trở nên hỗn loạn. ICONIX nhấn mạnh rằng Technical Architecture phải thể hiện rõ nơi nào chịu trách nhiệm cho các cơ chế bảo mật chính, thay vì để rải rác khắp nơi trong code.

Trong kiến trúc Internet Bookstore, tầng Presentation là nơi đầu tiên tiếp nhận input từ người dùng. Đây là nơi cần thực hiện các bước xác thực (authentication) và kiểm tra dữ liệu cơ bản (input validation). Tầng Business chịu trách nhiệm áp dụng các quy tắc kiểm soát truy cập (authorization) — ví dụ, chỉ người dùng đã đăng nhập mới được viết review, hoặc chỉ admin mới được xóa sách. Tầng Persistence không nên thực hiện kiểm tra quyền, nhưng phải được bảo vệ khỏi SQL injection, truy vấn không hợp lệ và đảm bảo mọi truy xuất đều đi qua DAO, không để UI gọi thẳng xuống database.

Một kiến trúc tốt cũng hỗ trợ áp dụng các cơ chế bảo mật tập trung, thay vì rải rác. Ví dụ, Spring Security có thể tích hợp trực tiếp vào controller layer để thực hiện authentication và authorization thống nhất. Các service lớp dưới không cần quan tâm đến chi tiết đăng nhập, nhưng vẫn được bảo vệ gián tiếp thông qua context bảo mật được truyền xuống. IoC/DI một lần nữa đóng vai trò hỗ trợ: các filter bảo mật, service

xác thực hoặc context security có thể được inject vào controller, đảm bảo tính tách biệt và kiểm thử được.

Ngoài ra, kiến trúc còn có thể giúp giảm thiểu tấn công bằng cách giới hạn luồng giao tiếp giữa các tầng. Khi Presentation không thể gọi trực tiếp xuống Persistence, kẻ tấn công cũng khó lợi dụng lỗ hổng injection để truy cập dữ liệu trái phép. Việc buộc mọi truy xuất phải đi qua Business Layer cho phép nhóm kiểm soát quyền truy cập ở một điểm duy nhất, dễ audit và dễ vá lỗi nếu có vấn đề.

3. Sự giao thoa giữa Testability và Security

Một điều thú vị là testability và security không tách biệt nhau, mà thường hỗ trợ lẫn nhau. Một kiến trúc dễ kiểm thử cũng thường dễ kiểm soát bảo mật, vì nó có ranh giới rõ ràng, interface minh bạch và ít coupling. Khi các tầng được mô-đun hóa tốt, nhóm có thể dễ dàng viết test cho các luồng bảo mật: kiểm thử authentication, kiểm thử quyền truy cập use case, kiểm thử phản ứng với dữ liệu độc hại. Ngược lại, một kiến trúc “rối dây” sẽ khiến cả test lẫn bảo mật trở thành ác mộng.

ICONIX coi việc đưa testability và security vào giai đoạn Technical Architecture là một bước chuyển tư duy quan trọng: thay vì xem đây là “việc của giai đoạn cuối”, kiến trúc trở thành nơi thiết lập điều kiện tiên quyết để kiểm thử và bảo mật có thể triển khai hiệu quả về sau.

7.6 Top 10 Technical Architecture Errors

Một kiến trúc kỹ thuật yếu thường không bộc lộ lỗi ngay lập tức. Lúc đầu, hệ thống có thể vẫn chạy được, thậm chí phát triển nhanh vì “không phải suy nghĩ nhiều về kiến trúc”. Nhưng về sau, khi số lượng use case tăng, nhóm phát triển mở rộng, và yêu cầu thay đổi liên tục, những lỗi kiến trúc ban đầu sẽ bộc lộ rõ rệt: code trở nên chằng chéo, khó kiểm thử, bảo trì tốn kém, và mỗi thay đổi nhỏ đều có thể gây hiệu ứng domino. ICONIX tổng kết mười lỗi kiến trúc phổ biến để giúp các nhóm nhận diện và tránh sớm.

1. Thiếu phân tầng kiến trúc rõ ràng

Đây là lỗi căn bản và cũng phổ biến nhất. Nhiều nhóm bắt đầu lập trình mà không xác định rõ Presentation, Business và Persistence Layer. UI có thể gọi trực tiếp xuống database, logic nghiệp vụ rải rác trong JSP, và không có ranh giới trách nhiệm rõ ràng. Kết quả là code trở thành “mì spaghetti”, không thể kiểm thử độc lập, thay đổi ở một nơi ảnh hưởng dây chuyền khắp nơi. Trong Internet Bookstore, nếu trang BookDetailsPage.jsp tự truy vấn SQL, không thông qua controller và DAO, đó chính là biểu hiện của lỗi này.

2. Vi phạm nguyên tắc phụ thuộc một chiều

Một kiến trúc đúng phải có luồng phụ thuộc theo một chiều rõ ràng: Presentation → Business → Persistence. Lỗi xảy ra khi tầng dưới gọi ngược lên tầng trên, hoặc các tầng tạo vòng phụ thuộc lẫn nhau. Ví dụ, một DAO gọi ngược lại service để kiểm tra

quyền truy cập, hoặc controller trực tiếp thao tác database. Những vòng phụ thuộc này khiến hệ thống khó mở rộng, test phức tạp và dễ sinh lỗi tiềm ẩn.

3. Trộn lẫn quá nhiều công nghệ và pattern

Một lỗi khá phổ biến trong các dự án sinh viên là cố gắng “nhét” thật nhiều framework, pattern và công nghệ để hệ thống trông “xịn”. Ví dụ, vừa dùng Spring MVC, vừa microservices, vừa event-driven architecture... mà không có lý do rõ ràng. Kết quả là kiến trúc trở nên rối rắm, khó học, khó debug, và các thành viên mới khó theo kịp. ICONIX khuyên nên chọn một kiến trúc đơn giản, phù hợp với yêu cầu dự án và triển khai nhất quán.

4. Không liên kết kiến trúc với quyết định kỹ thuật thực tế

Một lỗi tinh vi hơn là thiết kế kiến trúc “đẹp trên giấy” nhưng không phản ánh các framework, công nghệ và ràng buộc hạ tầng thực tế. Ví dụ, vẽ kiến trúc ba tầng chuẩn nhưng trong code lại không sử dụng Spring MVC, không có DI container, hoặc deployment khác hẳn sơ đồ. Đây là lỗi khiến kiến trúc trở nên “vô nghĩa” vì không dẫn dắt được triển khai. Internet Bookstore tránh được lỗi này bằng cách dùng Spring MVC và thể hiện rõ điều đó trong kiến trúc.

5. Không áp dụng hoặc lạm dụng Dependency Injection

Dependency Injection là chìa khóa để giảm coupling giữa các tầng. Lỗi phổ biến là không dùng DI, để các lớp tự khởi tạo dependency (new lung tung), dẫn đến kiến trúc chỉ tồn tại trên sơ đồ. Ở chiều ngược lại, một số nhóm lạm dụng DI bằng cách tiêm quá nhiều dependency hoặc cấu hình container quá phức tạp, khiến việc debug và kiểm thử trở nên khó khăn. Điều quan trọng là sử dụng DI đúng mức, rõ ràng, và nhất quán.

6. Không chú ý đến testability trong thiết kế kiến trúc

Khi kiến trúc không tách biệt rõ các tầng, không có interface rõ ràng, và không áp dụng IoC/DI, kiểm thử sẽ trở nên gần như không thể. Các nhóm thường phát hiện ra điều này quá muộn, khi muốn viết test mà code đã đan chéo phức tạp. Đây là lỗi cực kỳ nguy hiểm vì nó ảnh hưởng trực tiếp đến chất lượng dài hạn của dự án. Internet Bookstore tránh được điều này bằng kiến trúc tách tầng rõ ràng và dependency injection xuyên suốt.

7. Bỏ qua các khía cạnh bảo mật ở tầng kiến trúc

Một số nhóm sinh viên tập trung toàn bộ vào chức năng mà quên mất rằng bảo mật cũng cần bắt đầu từ kiến trúc. Việc không xác định rõ nơi kiểm tra quyền, không chuẩn hóa luồng xử lý xác thực, hoặc để input không được kiểm tra lọt xuống tầng persistence là những lỗi phổ biến. Khi hệ thống mở rộng, các lỗ hổng này trở nên rất khó vá vì chúng nằm rải rác, không tập trung. Kiến trúc tốt phải xác định rõ ràng điểm kiểm soát bảo mật tập trung.

8. Không bảo trì kiến trúc như một artefact sống

Một lỗi nguy hiểm nhưng thường bị xem nhẹ: sau khi thiết kế xong kiến trúc, nhóm coi tài liệu kiến trúc như “xong nhiệm vụ” và không bao giờ cập nhật nữa. Trong khi đó, code thì thay đổi liên tục, dẫn đến tài liệu kiến trúc “chết”, không còn phản ánh thực tế. Kết quả là kiến trúc mất vai trò định hướng, sequence diagram trở nên lệch pha, và onboarding thành viên mới gặp khó khăn. ICONIX khuyến nghị kiến trúc phải được cập nhật cùng với các mô hình khác khi có thay đổi quan trọng.

9. Không kiểm duyệt kiến trúc sớm trong nhóm

Một kiến trúc tốt cần được mọi người hiểu và cam kết thực hiện, nhưng nhiều nhóm lại không tổ chức bất kỳ buổi review kiến trúc nào. Điều này dẫn đến việc mỗi thành viên hiểu và triển khai khác nhau, tạo ra kiến trúc “trên giấy một kiểu, trong code một nẻo”. ICONIX nhấn mạnh việc tổ chức một buổi “architecture walkthrough” trước khi bước vào thiết kế chi tiết là bắt buộc để đảm bảo sự thống nhất.

10. Thiếu traceability từ use case xuống kiến trúc

Cuối cùng, một lỗi mang tính hệ thống là không đảm bảo được “đường đi rõ ràng” của từng use case xuyên qua kiến trúc. Khi không có traceability, sequence diagram trở nên rời rạc, khó mapping với kiến trúc, và hệ thống mất tính nhất quán logic. Kiến trúc đúng phải cho phép bạn “lần ngược” từ một use case cụ thể để thấy nó đi qua tầng nào, component nào, interface nào. Đây là một trong những nguyên tắc cốt lõi của ICONIX.

Tổng kết

Chương này nhấn mạnh rằng Technical Architecture đóng vai trò là chiếc cầu nối quyết định giữa giai đoạn phân tích và thiết kế chi tiết, đảm bảo các mô hình use case, domain model và robustness có thể được hiện thực hóa một cách logic, thống nhất và khả thi về mặt kỹ thuật. Một kiến trúc tốt không chỉ tổ chức hệ thống thành các tầng rõ ràng mà còn hỗ trợ kiểm thử, bảo mật, khả năng mở rộng và bảo trì về sau. Thông qua ví dụ Internet Bookstore, ta thấy cách kiến trúc ba tầng, cùng cơ chế IoC/DI, tạo nên một khung kỹ thuật chặt chẽ giúp hiện thực hóa luồng use case một cách tuyến tính, dễ kiểm soát. Những nguyên tắc như giữ kiến trúc đơn giản, phân tầng rõ ràng, gắn với quyết định kỹ thuật thực tế và chia sẻ sớm trong nhóm đóng vai trò nền tảng. Đồng thời, việc xem xét testability và security ngay từ giai đoạn kiến trúc giúp dự án tránh được các rủi ro lớn khi mở rộng. Kiến trúc cũng không phải là tài liệu tĩnh mà cần được duy trì song song với quá trình phát triển để giữ vai trò định hướng. IoC và DI minh chứng cho việc kiến trúc không chỉ nằm trên giấy mà còn thể hiện qua các cơ chế thực thi trong code. Các lỗi kiến trúc phổ biến mà ICONIX nêu ra phản ánh những sai lầm thực tiễn mà các nhóm cần tránh, từ việc không phân tầng rõ ràng đến thiếu traceability. Hiểu đúng vai trò và thiết kế cẩn trọng Technical Architecture sẽ giúp toàn bộ dự án phát triển nhanh hơn, ít lỗi hơn và dễ bảo trì hơn. Đây chính là nền móng để chuyển sang thiết kế sequence diagram và triển khai hệ thống hiệu quả.

Câu hỏi ôn tập

1. Giải thích vai trò của Technical Architecture trong quy trình ICONIX.
2. Tại sao kiến trúc phân tầng lại quan trọng đối với khả năng mở rộng và kiểm thử?
3. IoC và DI giúp giảm coupling giữa các tầng kiến trúc như thế nào?
4. Trình bày luồng xử lý của use case “Show Book Details” qua ba tầng kiến trúc.
5. Nêu các lợi ích chính của việc sử dụng component diagram và deployment diagram trong kiến trúc.
6. Phân tích mối liên hệ giữa kiến trúc kỹ thuật và testability.
7. Vì sao security phải được xem xét ngay từ giai đoạn kiến trúc?
8. Kể tên và giải thích ngắn gọn ba lỗi kiến trúc phổ biến mà sinh viên hay mắc phải.
9. Giải thích khái niệm traceability từ use case xuống kiến trúc và tại sao nó quan trọng.
10. Theo bạn, điều gì khiến Technical Architecture trở thành một artefact “sống” trong suốt vòng đời phát triển phần mềm?